Silo System on VFR Hardware — Technical Mapping Specification v1.0

Mapping Silo Data-Only Architecture to the Integer ALU Based Computing ISA

Registry: [?]

Series Path: [CKS-0-2026] → [CKS-MATH-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-10-2026] → [CKS-MATH-104-2026] → [?] → [?]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: March 11, 2026

Domain: Machine Learning / Integer Arithmetic / Neural Network Training

Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Classification: Theory of Everything from First Principles

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological "Truth" is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google's Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Lexicon: [CKS-LEX-12-2026]

---

1. Core Principle

The Silo data-only execution system and the VFR integer processor share a single design axiom: everything is uniform data processed by uniform operations. Silo eliminates the wall between code and data in software. VFR eliminates the wall between code and data in hardware. The mapping between them is direct.

Every Silo concept — Entity, StateMachine, Prolog, UtilityAI, LogicBlock, Scene, Networking — maps to VFR batch streams processed by the existing ISA. No new instructions are required. No extensions to the architecture. The 35-instruction integer ALU already contains everything needed to run the complete Silo stack.

---

2. Entity as VFR Batch

2.1 The Universal Container

A Silo Entity is a fixed-size struct with ~20 subsystems, all optional, all defaulted. Every entity has the same layout regardless of what it represents.

On VFR hardware, an Entity is a single VFR value at a known depth. The depth corresponds to the number of fields. All entities in a scene share the same depth, so they form a uniform batch.

2.2 Batch Representation

Entity batch:

[F=entity_scale, count=10000, depth=48]

Each value (depth 48 = 48 VFR pairs):

[id, 0] pair 0: entity ID

[type, 0] pair 1: entity type (enum as i32)

[sm_id, 0] pair 2: state machine ID

[state, 0] pair 3: current state (state ID as i32)

[pos_x, rx] pair 4: position X

[pos_y, ry] pair 5: position Y

[health, 0] pair 6: health

[max_health, 0] pair 7: max health

[speed, rs] pair 8: movement speed

[target_id, 0] pair 9: current target entity index

[damage, 0] pair 10: combat damage

… remaining subsystem fields

[is_active, 0] pair 47: active flag (0 or 1)

F encodes the domain scale. Count is the number of entities. Depth is the number of fields per entity. Every entity occupies exactly `depth $\times$ 5` bytes. The batch is a flat array with fixed stride.

2.3 Field Access

Accessing `entity[i].health` is a direct address calculation:

BLOAD [entity_batch_addr] ; load header: F, count=10000, depth=48

; To access entity i, field 6 (health):

; address = batch_data_start + (i × 48 × 5) + (6 × 5)

; Using batch control:

; set BI = i

; BADDR gives base of entity i

; add field offset (6 × 5 = 30 bytes)

No pointer chasing. No hash lookup. No field name resolution. A multiply and an add.

2.4 Field Replacement

Silo's field replacement (health becomes bank balance, stamina becomes credit limit) is zero-cost on VFR hardware. The data is identical — same i32 at the same offset. The replacement table is metadata consumed by the UI layer, not by the compute pipeline. The ALU never sees field names. It sees pair index 6 in a batch of depth 48.

2.5 The Dragon and the Chair

A dragon: entity batch entry where pairs 6 (health=500), 10 (damage=50), 8 (speed=3) have gameplay values.

A chair: entity batch entry where pairs 4-5 (position) and pair 1 (type) have values, everything else is default (0).

Same batch. Same depth. Same stride. Same processing. The ALU does not know dragons exist.

---

3. State Machine as Batch Stream

3.1 Structure

A Silo StateMachine is a graph of states with transitions. Each state references a behavior set. Each transition has conditions (event, rule, duration).

On VFR hardware, a state machine is a batch stream with two segments: a state batch and a transition batch.

3.2 State Batch

State batch:

[F=sm_states, count=5, depth=3]

Each state (depth 3):

[state_id, 0] pair 0: unique state identifier

[behavior_set_id, 0] pair 1: reference to behavior set batch

[force_action, 0] pair 2: forced action ID (-1 = none, use UAI)

3.3 Transition Batch

Transition batch:

[F=sm_transitions, count=12, depth=6]

Each transition (depth 6):

[from_state, 0] pair 0: source state ID

[to_state, 0] pair 1: target state ID

[event_type, 0] pair 2: event trigger (-1 = none)

[rule_id, 0] pair 3: prolog rule ID (-1 = none)

[duration_min, r] pair 4: minimum duration (VFR for fractional seconds)

[delete_entity, 0] pair 5: delete on transition (0 or 1)

3.4 State Machine Evaluation

For each entity, find valid transitions from current state:

; Entity's current state is in V1

; Transition batch loaded

BLOAD [transition_batch_addr]

loop:

JDONE no_transition

BADDR V0

LDV V2, [V0] ; load from_state of this transition

CMP V1, V2 ; does it match entity's current state?

JEQ check_conditions

BNEXT

JMP loop

check_conditions:

LDV V3, [V0 + 10] ; load event_type (pair 2)

LDV V4, [V0 + 15] ; load rule_id (pair 3)

; check event match, check rule (see Prolog section)

; if all pass:

LDV V5, [V0 + 5] ; load to_state

STV [entity + 15], V5 ; write new state to entity pair 3

BNEXT

JMP loop

no_transition:

; remain in current state, execute behavior set

This is a filter over the transition batch. Find rows where `from_state == current_state`, check conditions, apply first match. Pure integer compare and branch.

3.5 Multiple State Machines Per Entity

Entity fields include multiple state machine slots (behavior SM, animation SM, audio SM). Each is an independent state ID field in the entity batch. The same evaluation code runs on different field offsets. No special handling. Same batch, different pair index.

---

4. Prolog as Batch Filtering

4.1 The Key Insight

Prolog evaluation on VFR hardware is not a logic engine. It is batch filtering with integer comparison. No strings. No unification algorithm. No backtracking search. Just flat arrays of integers matched by value.

4.2 Facts as Batches

A fact set is a batch where F is the predicate ID and each value is one fact.

Predicate 47: “has_target” takes 2 args (entity_id, target_id)

[F=47, count=500, depth=2]

[42, 0] [17, 0] ; entity 42 targets entity 17

[43, 0] [17, 0] ; entity 43 targets entity 17

[44, 0] [22, 0] ; entity 44 targets entity 22

…

Predicate 23: “in_range” takes 3 args (entity_id, target_id, distance)

[F=23, count=500, depth=3]

[42, 0] [17, 0] [45, r] ; entity 42 to target 17, distance 45

[43, 0] [17, 0] [120, r] ; entity 43 to target 17, distance 120

…

Different predicates have different F values and different depths. The batch header IS the schema. No type tags on individual values. Position determines meaning.

4.3 Fact Generation Per Frame

Silo regenerates facts from entity state every frame. On VFR hardware, this is a transform from entity batches to fact batches:

entity_batch |> generate_facts -> fact_batches

Transform: for each entity, emit facts:

if health < max_health * 0.2:

append [entity_id, 0] to predicate F=health_critical batch

if target_id != -1:

append [entity_id, 0] [target_id, 0] to predicate F=has_target batch

…

This is an element-wise transform over the entity batch that produces multiple output batches (one per predicate). Each output batch has a different F and depth.

The fact generation transform is compiled once. It runs every frame over the entity batch. Its output is the current knowledge base — a stream of predicate-keyed batches.

4.4 Rule Evaluation as Chained Filters

A Prolog rule: `enemy_nearby :- has_target(X, Y), in_range(X, Y, D), D < 100`

This evaluates as three batch operations:

Step 1: Filter has_target batch (F=47) for this entity

[F=47, count=500, depth=2]

|> filter { pair[0].v == entity_id } -> matches_1

; result: target IDs for this entity

Step 2: Filter in_range batch (F=23) for matching entity+target pairs

[F=23, count=500, depth=3]

|> filter { pair[0].v == entity_id AND pair[1].v in matches_1.pair[1].v } -> matches_2

; result: distances to matched targets

Step 3: Filter matches_2 for distance < 100

matches_2

|> filter { pair[2].v < 100 } -> result

; result: non-empty means rule passes

Each step is a batch filter. Each filter is integer comparison across a flat array. SIMD-perfect. No unification. No backtracking. No string matching.

4.5 Variable Binding

In traditional Prolog, variables bind during unification. On VFR hardware, binding is reading column values from matched rows.

Query: `has_target(42, ?Y)` — find Y where entity 42 has a target.

Filter: pair[0].v == 42 in predicate F=47 batch

Result row: [42, 0] [17, 0]

Binding: Y = 17 (just read pair[1].v from the matched row)

No binding table. No environment stack. The matched row IS the binding. Read the column you need.

4.6 Mass Prolog Evaluation

Silo evaluates Prolog rules for every entity every frame. On VFR hardware, this is a batch operation over entities, not a per-entity serial evaluation.

Instead of: for each entity, evaluate rule, check facts…

Do: filter the fact batch for ALL entities simultaneously.

; “Which entities have health_critical?”

[F=health_critical, count=200, depth=1]

; This batch already IS the answer. Every entity ID in it has critical health.

; Checking if entity 42 has critical health = scan for 42 in this batch.

; Checking for ALL entities = the batch itself is the result set.

The fact generation step already partitions entities into predicate batches. Rule evaluation is set intersection between predicate batches. Set intersection on sorted integer arrays is a merge operation — linear time, SIMD-friendly, no branching.

4.7 Spatial Predicates

Silo’s Prolog supports vector2, rectangle, and circle terms for spatial queries. On VFR hardware:

Point-in-rectangle test:

px >= rect_x AND px <= rect_x + rect_w AND py >= rect_y AND py <= rect_y + rect_h

Four integer comparisons. CMP, JLT, JGT. No floating point.

Distance test (squared, avoids sqrt):

dx = ax - bx

dy = ay - by

dist_sq = dx*dx* + *dy*dy

dist_sq < threshold_sq

SUB, MUL, ADD, CMP. Five integer operations.

Circle containment:

dist_sq < radius * radius

Same as distance test with radius threshold.

All spatial predicates reduce to integer arithmetic and comparison. The VFR remainder gives sub-unit precision for positions. No floating-point geometry library needed.

---

5. Utility AI as Batch Scoring

5.1 Behavior Scoring

Silo's UtilityAI scores behaviors by multiplying normalized consideration values. On VFR hardware, this is an element-wise transform producing a score batch.

5.2 Consideration as Transform

Each consideration reads a value, normalizes it to a range, applies a curve, and produces a score.

Consideration: "health_ratio"

input: entity.health (pair 6)

range: [0, max_health]

curve: linear

VFR implementation:

LDV V1, [entity + 30] ; load health (pair 6)

LDV V2, [entity + 35] ; load max_health (pair 7)

; normalize: score = health * 256 / max_health (fixed-point in base 32)

SHL V1, V1, 8 ; health * 256 for fixed-point precision

; integer division by max_health via reciprocal multiply

MUL V3, V1, V4 ; V4 = precomputed reciprocal of max_health

SHR V3, V3, 8 ; shift back

; V3 now holds normalized score 0-256 (fixed-point)

Curves are lookup tables or piecewise integer functions:

Linear: score = input (identity)

Inverse: score = 256 - input

Quadratic: score = (input * input) » 8

Boolean: score = (input > 128) ? 256 : 0

Sigmoid: score = sigmoid_table[input] (256-entry lookup table)

All integer. All single-cycle operations except the multiply.

5.3 Multiplicative Scoring

running_score = 256 (fixed-point 1.0)

For each consideration:

compute consideration score (0-256)

running_score = (running_score * consideration_score) » 8

If any consideration = 0, running_score = 0 (fail-fast)

; Start with score = 256 (1.0 in fixed-point)

LDV V10, [immediate_256]

; Consideration 1: check prolog rule (batch filter)

; if rule fails, score = 0

; … filter operation, sets FAIL flag if no matches …

TFAIL score_zero

; Consideration 2: health ratio

; V3 holds normalized score 0-256

MUL V10, V10, V3

SHR V10, V10, 8 ; maintain fixed-point scale

; Consideration 3: distance score

; V5 holds normalized score 0-256

MUL V10, V10, V5

SHR V10, V10, 8

; V10 holds final behavior score

JMP store_score

score_zero:

; Any consideration failed, entire behavior = 0

LDV V10, [immediate_0]

store_score:

; store to behavior score array

The fail-fast on zero is a single `CMP` + `JEQ` after each consideration multiply. One zero kills the behavior immediately. No wasted cycles scoring remaining considerations.

5.4 Behavior Selection

After scoring all behaviors, selection is finding the maximum in a score array:

Behavior scores batch:

[F=scores, count=5, depth=2]

[score_0, behavior_id_0] [score_1, behavior_id_1] …

Find max:

LDV V8, [immediate_0] ; best score = 0

LDV V9, [immediate_neg1] ; best index = -1

BLOAD [scores_addr]

loop:

JDONE done

BADDR V0

LDV V1, [V0] ; load score

CMP V1, V8 ; better than current best?

JGT new_best

BNEXT

JMP loop

new_best:

LDV V8, V1 ; update best score

LDV V9, [V0 + 5] ; update best behavior ID

BNEXT

JMP loop

done:

; V9 holds winning behavior ID

Linear scan, integer comparison, no sorting needed. For small behavior sets (3-8 behaviors typical), this is a few dozen cycles.

5.5 Mass Scoring Across Entities

Score all behaviors for all entities simultaneously:

; For 10,000 entities with 5 behaviors each:

; Partition entities across cores

; Each core scores all behaviors for its entity subset

; No cross-entity dependencies

; Pure SIMD within each core

; Core 0: entities 0-999, score 5 behaviors each = 5000 score operations

; Core 1: entities 1000-1999, same

; …

; All cores run simultaneously, no synchronization until done

This is the batch model in action. UtilityAI scoring is embarrassingly parallel across entities.

---

6. Logic Blocks as Instruction Batches

6.1 Stack Machine on VFR

Silo's LogicBlocks are a stack-based bytecode interpreter. On VFR hardware, a LogicBlock stack is literally an instruction batch.

Each block type maps to one or two ISA instructions:

Silo BlockType VFR ISA

——————————— ———————

SetValueInt → STV (store to entity field)

SetValueFloat → STVR (store VFR pair to entity field)

GetValueInt → LDV (load from immediate)

GetValueFloat → LDVR (load VFR pair from immediate)

DrGetFloat → LDV at computed field offset

DrGetInt → LDV at computed field offset

MathAdd → ADD

MathMultiply → MUL

MathSub → SUB

LogicAnd → AND

LogicOr → OR

LogicEqual → CMP + conditional

LogicLess → CMP + JLT

If → CMP + JEQ/JLT/JGT

Else → JMP (skip to else block)

While → JMP (back to loop start)

ExecutePrologQuery → batch filter sequence (see Section 4)

ExecuteCommand → jump to command handler batch

6.2 Block Stack as Code Batch

A Silo LogicBlock stack:

zig

DrGetFloat("target.health") // push target health

GetValueFloat(50.0) // push 50

MathSub // subtract

DrSetFloat("target.health") // store result

Compiles to a VFR code batch:

[F=code, count=4, depth=1]

[LDV_target_health, 0] ; instruction: load target health to V1

[LDV_immediate_50, 0] ; instruction: load 50 to V2

[SUB_V1_V1_V2, 0] ; instruction: V1 = V1 - V2

[STV_target_health, 0] ; instruction: store V1 to target health

Code IS data. The instruction batch has the same format as any data batch. F identifies it as code. The core processes it identically — it reads VFR pairs and executes them.

6.3 Path-Based Field Access

Silo's `DrGetFloat("character.health")` resolves a string path to a field offset at runtime. On VFR hardware, the compiler resolves this to a pair index at compile time.

"character.health" → pair index 6 → byte offset 30

DrGetFloat("character.health") compiles to: LDV V1, [entity_base + 30]

No string parsing. No hash lookup. No runtime resolution. The compiler knows the entity depth layout and computes the offset. The instruction is a direct memory load.

6.4 Type Safety Without Runtime Checks

Silo ensures logic block type safety through UI validation — only compatible blocks can be connected. On VFR hardware, type safety is structural. A batch has a declared depth. Every value in the batch has the same structure. If the compiler emits a load from pair index 6 and pair index 6 is defined as health, the types match by construction.

Invalid paths (field doesn't exist, wrong type) are compile-time errors. The runtime never encounters a type mismatch because the compiler verified all offsets against the entity depth layout.

---

7. Envelopes as Time-Bounded Batch Transforms

7.1 Envelope Representation

A Silo envelope modifies a stat over time with a curve. On VFR hardware:

Envelope batch:

[F=envelopes, count=200, depth=7]

Each envelope (depth 7):

[source_entity, 0] pair 0: who created this effect

[target_entity, 0] pair 1: who receives the effect

[stat_offset, 0] pair 2: which pair index in entity to modify

[modifier, rm] pair 3: modification amount (VFR for precision)

[curve_type, 0] pair 4: curve ID (linear, quadratic, sigmoid…)

[time_start, rt] pair 5: start time (VFR for sub-frame precision)

[time_end, re] pair 6: end time

7.2 Envelope Processing

Every frame, process all active envelopes:

```
BLOAD [envelope_batch_addr]
loop:
JDONE done
BADDR V0
; Load envelope fields
LDV V1, [V0 + 25] ; time_start (pair 5)
LDV V2, [V0 + 30] ; time_end (pair 6)
LDV V3, [current_time] ; current frame time
; Check if envelope is active
CMP V3, V1
JLT skip ; not started yet
CMP V3, V2
JGT expire ; past end time
; Compute progress: t = (current - start) / (end - start)
SUB V4, V3, V1 ; current - start
SUB V5, V2, V1 ; end - start
SHL V4, V4, 8 ; fixed-point precision
MUL V4, V4, V6 ; V6 = precomputed reciprocal of duration
SHR V4, V4, 8 ; t in 0-256 range
; Apply curve (lookup table or integer function)
; … curve application …
; Apply modifier to target entity's stat
LDV V7, [V0 + 5] ; target entity index
LDV V8, [V0 + 10] ; stat pair offset
; compute entity address + stat offset
; load current stat, add curved modifier, store back
skip:
BNEXT
```

JMP loop

expire:

; mark envelope as inactive (set time_end to 0 or remove from batch)

BNEXT

JMP loop

done:

Envelopes are a batch transform. Process all active envelopes in one pass. No per-entity envelope lists. No pointer chasing. One flat batch of all envelopes in the scene.

---

8. Scene Isolation as Core Memory Isolation

8.1 Direct Mapping

Silo's Scene isolation maps one-to-one to VFR core memory isolation.

Silo concept VFR hardware

Scene → Core (or group of cores)

Scene.actors[] → Entity batch in core's local memory

Scene.state_machines → SM batches in core's local memory

Scene.behavior_sets → Behavior batches in core's local memory

Scene data isolation → Core cannot access another core's memory

Cross-scene transfer → DMA TSEND/TRECV between cores

8.2 Access Control

Silo's SceneSetItem with `allow_read_scene_ids` and `allow_write_paths` maps to DMA permissions in the host controller:

Scene 0 (network) can DMA to Scene 1 (game):

only to offsets corresponding to "network.inbound" fields

host controller enforces: TSEND from core 0 to core 1 only at allowed offset ranges

Scene 2 (inspector) can DMA-read from Scene 1 (game):

only from offsets corresponding to "actors.*.transform" and "actors.*.character"

host controller enforces: TRECV by core 2 from core 1 only at allowed offset ranges

The host controller maintains a permission table:

Permission batch:

[F=permissions, count=N, depth=5]

Each permission:

[from_core, 0] pair 0: source core

[to_core, 0] pair 1: destination core

[offset_start, 0] pair 2: allowed memory range start

[offset_end, 0] pair 3: allowed memory range end

[read_write, 0] pair 4: 0=read, 1=write, 2=both

DMA transfers are checked against this permission batch before execution. Unauthorized transfers are dropped. This is hardware-enforced — a core cannot bypass the host controller's permission check.

8.3 Default Deny

Empty permission batch = no cross-core transfers allowed. Every permission must be explicitly granted. This matches Silo's default-deny scene isolation.

---

9. Network Processing as Batch Pipeline

9.1 Fixed-Size Packets as VFR Batches

Silo's InboundPacket is a 2048-byte fixed-size struct. On VFR hardware:

Packet batch:

[F=packet, count=64, depth=410]

Each packet (depth 410 = 410 VFR pairs = 2050 bytes):

[src_mac_hi, 0] pair 0: source MAC high bytes

[src_mac_lo, 0] pair 1: source MAC low bytes

[dst_mac_hi, 0] pair 2: destination MAC

[dst_mac_lo, 0] pair 3: destination MAC

[ethertype, 0] pair 4: protocol type

[src_ip, 0] pair 5: source IP

[dst_ip, 0] pair 6: destination IP

[src_port, 0] pair 7: source port

[dst_port, 0] pair 8: destination port

…

[payload_0-3, 0] pair 9+: payload as i32 words

…

[payload_len, 0] pair 409: actual payload length

64 packets, fixed stride, uniform depth. The network core processes them as a standard batch.

9.2 Protocol Dispatch

Silo routes packets by EtherType and protocol number. On VFR hardware, this is batch filtering:

; Route by EtherType

packet_batch

|> filter { pair[4].v == 0x0806 } -> arp_batch ; ARP |> filter { pair[4].v == 0x0800 } -> ip_batch ; IPv4 ; Route IP by protocol

ip_batch

|> filter { pair[protocol].v == 1 } -> icmp_batch ; ICMP |> filter { pair[protocol].v == 6 } -> tcp_batch ; TCP |> filter { pair[protocol].v == 17 } -> udp_batch ; UDP ; Route UDP by port

udp_batch

|> filter { pair[8].v == 53 } -> dns_batch ; DNS |> filter { pair[8].v == 68 } -> dhcp_batch ; DHCP Each filter produces a sub-batch. Each sub-batch is dispatched to the appropriate handler — which is itself a batch transform. The entire network stack is a tree of batch filters and transforms.

9.3 Input Isolation

Silo's geometric security (fixed packets, path-based write control) is hardware-native:

1. NIC DMA writes fixed-size batch to network core's memory (cannot overflow, fixed depth)
2. Network core filters and validates (batch operations)
3. Network core TSEND allowed fields to game core (DMA, permission-checked)
4. Game core receives only whitelisted fields at whitelisted offsets
5. Game core cannot TSEND to network core's output buffer (no write permission)

Buffer overflow is impossible — batch depth is fixed. Code injection is impossible — data is VFR pairs processed as integers, never executed. Privilege escalation is impossible — DMA permissions are hardware-enforced.

9.4 Malicious Input Handling

Bad values in allowed fields (NaN equivalent, extreme values) are handled by the next frame's Prolog evaluation:

; Fact generation checks input validity

; input.mouse_x loaded, compared against screen bounds

CMP V1, V_screen_min

JLT invalid_input

CMP V1, V_screen_max

JGT invalid_input

; only valid: append to valid_input predicate batch

Invalid input never enters the predicate batch. Behaviors that depend on valid input score 0 in UtilityAI. Entity continues previous behavior. Next frame, new input arrives. One frame of ignored input is 16ms. Undetectable.

---

10. Trace System as Batch Recording

10.1 Trace Data

Silo captures every decision per entity per frame. On VFR hardware, tracing is writing batch snapshots.

Trace batch per frame:

[F=trace, count=10000, depth=20]

Each trace entry (per entity):

[entity_id, 0] pair 0

[frame_number, 0] pair 1

[state_before, 0] pair 2

[state_after, 0] pair 3

[behavior_0_score, r] pair 4

[behavior_1_score, r] pair 5

[behavior_2_score, r] pair 6

[behavior_3_score, r] pair 7

[behavior_4_score, r] pair 8

[selected_behavior,0] pair 9

[stat_health_before,0] pair 10

[stat_health_after, 0] pair 11

…

Tracing is appending to a trace batch during processing. Each score, each state transition, each stat change writes a VFR pair to the trace entry. At frame end, the trace batch is complete.

10.2 Trace Queries

"Why did entity 42 attack at frame 1534?" is a batch filter:

trace_batch |> filter { pair[0].v == 42 AND pair[1].v == 1534 } -> result

; Read result: behavior scores in pairs 4-8, selected in pair 9

One filter operation. Immediate answer. No log parsing. No stack trace reconstruction.

10.3 Frame Replay

Replay frame 1534 with modified rules:

1. Load entity state snapshot from trace at frame 1533 (previous frame)
2. Load modified rule batches
3. Regenerate fact batches from entity state
4. Run state machine evaluation
5. Run Prolog filtering with new rules
6. Run UtilityAI scoring
7. Compare new trace output with original frame 1534 trace

All batch operations. A/B comparison is element-wise subtraction between two trace batches. Differences appear as non-zero values.

---

11. The Complete Frame Pipeline

11.1 One Frame on VFR Hardware

Host controller dispatches frame batch stream to cores:

Batch 1: Generate facts from entities

entity_batch |> fact_generation_transforms -> fact_batches[]

(Multiple output batches, one per predicate)

Batch 2: Evaluate state machine transitions

```
entity_batch + fact_batches + transition_batch
|> filter matching transitions
|> apply first valid transition per entity
-> updated entity_batch (state fields modified)
Batch 3: Determine active behavior sets
entity_batch |> lookup behavior_set_id from current state
-> behavior_assignment_batch
Batch 4: Score all behaviors (UtilityAI)
entity_batch + fact_batches + behavior_batches
|> score considerations (Prolog filters + curve math)
|> multiply scores
|> select winners
-> winning_behavior_batch
Batch 5: Execute winning behaviors (LogicBlocks)
entity_batch + winning_behavior_batch
|> execute logic block instruction batches
-> modified entity_batch
Batch 6: Process envelopes
entity_batch + envelope_batch + current_time
|> apply active envelope modifiers
-> final entity_batch
Batch 7: Record trace
all intermediate results -> trace_batch
Batch 8: Network I/O (parallel with above on dedicated core)
packet_batch |> filter |> validate |> route -> input fields
[0, 0, 0] end of frame
```

11.2 Parallelism

Batches 1-6 are the entity processing pipeline. Within each batch, entities are independent and can be partitioned across cores. The host controller splits each batch's count across available cores, DMAs the data, and waits for completion before the next batch.

Batch 8 (network) runs on a dedicated core in parallel with entity processing. It only touches entity input fields via permission-controlled DMA.

11.3 Timing Budget

At 60 FPS, frame budget is 16.67ms. The pipeline is 7 sequential batch passes over entity data. Each pass is partitioned across cores.

For 10,000 entities on 100 cores:

- 100 entities per core per batch pass
- Each entity processes in ~100 cycles (load, compare, arithmetic, store)
- 100 entities $\times$ 100 cycles = 10,000 cycles per pass
- 7 passes $\times$ 10,000 cycles = 70,000 cycles
- At 1 GHz: 70 microseconds per frame
- Headroom: 16,600 microseconds unused (99.6% budget remaining)

This leaves massive headroom for deeper Prolog evaluation, more complex behaviors, more entities, or lower clock speeds.

---

12. Compiler Responsibilities

12.1 Entity Layout

The compiler assigns pair indices to entity fields. This layout is fixed for the lifetime of a program:

compile time:

entity.id → pair 0

entity.type → pair 1

entity.state_machine_id → pair 2

entity.state → pair 3

entity.pos_x → pair 4

entity.pos_y → pair 5

entity.health → pair 6

…

All DrGet/DrSet paths resolved to pair indices.

All field accesses become direct offset calculations.

12.2 Predicate Assignment

The compiler assigns integer IDs to predicates:

compile time:

“has_target” → F = 47

“in_range” → F = 23

“health_critical” → F = 88

“has_weapon” → F = 12

…

All Prolog rule references become integer F values.

All fact batch headers use integer predicates.

12.3 Behavior Scoring Compilation

The compiler pre-computes:

- Reciprocals for normalization ranges (avoid runtime division)
- Curve lookup tables (256-entry i32 arrays)
- Consideration weight fixed-point values

12.4 Batch Fusion

The compiler fuses adjacent transforms where possible:

Before fusion:

Batch 1: load entity, generate fact A

Batch 2: load entity, generate fact B

Batch 3: load entity, generate fact C

After fusion:

Batch 1: load entity, generate facts A+B+C in one pass

Fewer passes over entity data = fewer memory reads = faster frames.

---

13. Summary

Silo Concept	VFR Representation	ISA Operations Used
Entity	Batch at fixed depth, one pair per field	LDVR, STVR, BLOAD, BADDR
State Machine	State batch + transition batch	CMP, JEQ, LDV, STV
Prolog Facts	Batch per predicate, F = predicate ID	Filter: CMP, JEQ, BNEXT
Prolog Rules	Chained batch filters	CMP, JEQ, JLT, JGT, BNEXT
Prolog Variables	Column read from matched row	LDV at column offset
UtilityAI Score	MUL chain with fixed-point normalization	MUL, SHR, CMP, JEQ
UtilityAI Select	Max scan over score array	CMP, JGT, LDV
Logic Blocks	Code batch (instructions as VFR pairs)	All ALU ops
Envelopes	Time-bounded modifier batch	SUB, MUL, SHR, CMP, ADD
Scene Isolation	Core memory isolation	TSEND, TRECV (permission-checked)
Network Packets	Fixed-depth batch, filter by protocol	CMP, JEQ, filter pattern
Trace	Append-only batch per frame	STVR during processing
Frame Replay	Reload snapshot batch, re-run pipeline	All ops (same as live frame)

No new instructions required. The 35-instruction ISA covers the complete Silo execution model. The architecture is sufficient. The intelligence is in the data.

Silo System on VFR Hardware — Technical Mapping Specification v1.0

Companion document to the ISA Specification v1.0 and Compiler Specification v1.0

[**CKS-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C MATH-0-2026

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026

[**CKS-MATH-10-2026**] Grand Unification. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-10-2026

[**CKS-MATH-104-2026**] Grand Unification v23. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-104-2026

[**CKS-LEX-12-2026**] Measurement Systems in the $\mathbb{Q}$-Substrate. Github: https://github.com/ghowland/cks/tree/main/papers/LEX-12-2026